

Lab07

Hadia Beshir

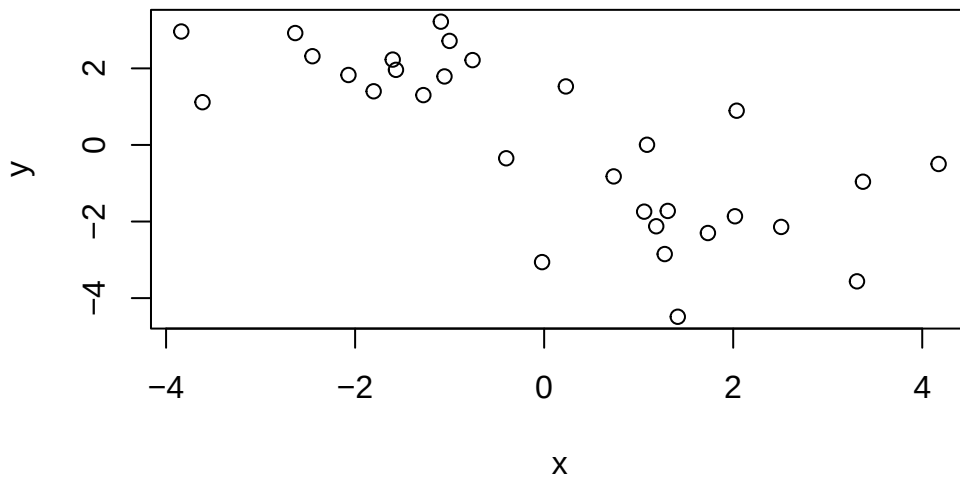
In class lesson

Generate some data

```
tmp_x <- c(rnorm(15, -2), rnorm(15, 2))  
tmp_y <- c(rnorm(15, 2), rnorm(15, -2))  
df <- cbind(x=tmp_x, y=tmp_y)
```

Plot the data

```
plot(df)
```



Lets do a k-means clustering analysis

```
km <- kmeans(df, 2, nstart=20)
km
```

K-means clustering with 2 clusters of sizes 16, 14

Cluster means:

	x	y
1	1.674450	-1.723549
2	-1.752789	2.107957

Clustering vector:

```
[1] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1 1 1 1 1
```

Within cluster sum of squares by cluster:

```
[1] 51.53292 21.61946
(between_SS / total_SS = 73.0 %)
```

Available components:

[1]	"cluster"	"centers"	"totss"	"withinss"	"tot.withinss"
[6]	"betweenss"	"size"	"iter"	"ifault"	

```
attributes(km)
```

\$names					
[1]	"cluster"	"centers"	"totss"	"withinss"	"tot.withinss"
[6]	"betweenss"	"size"	"iter"	"ifault"	

```
$class
[1] "kmeans"
```

Q: Number of points per a cluster ?

```
km$size
```

```
[1] 16 14
```

Q: cluster size ? above Q: Cluster assignment?

```
km$cluster
```

```
[1] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

```
table(km$cluster)
```

```
1 2  
16 14
```

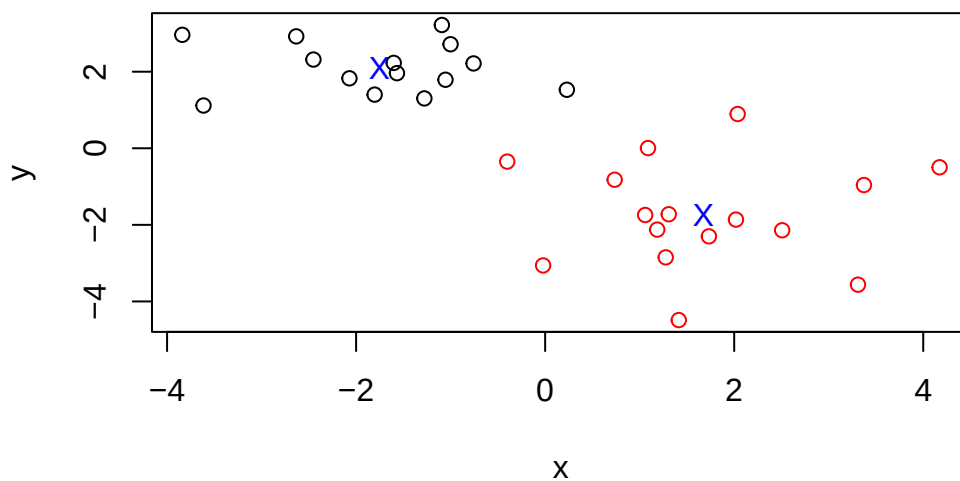
Q: Cluster center ?

```
km$centers
```

```
      x      y  
1  1.674450 -1.723549  
2 -1.752789  2.107957
```

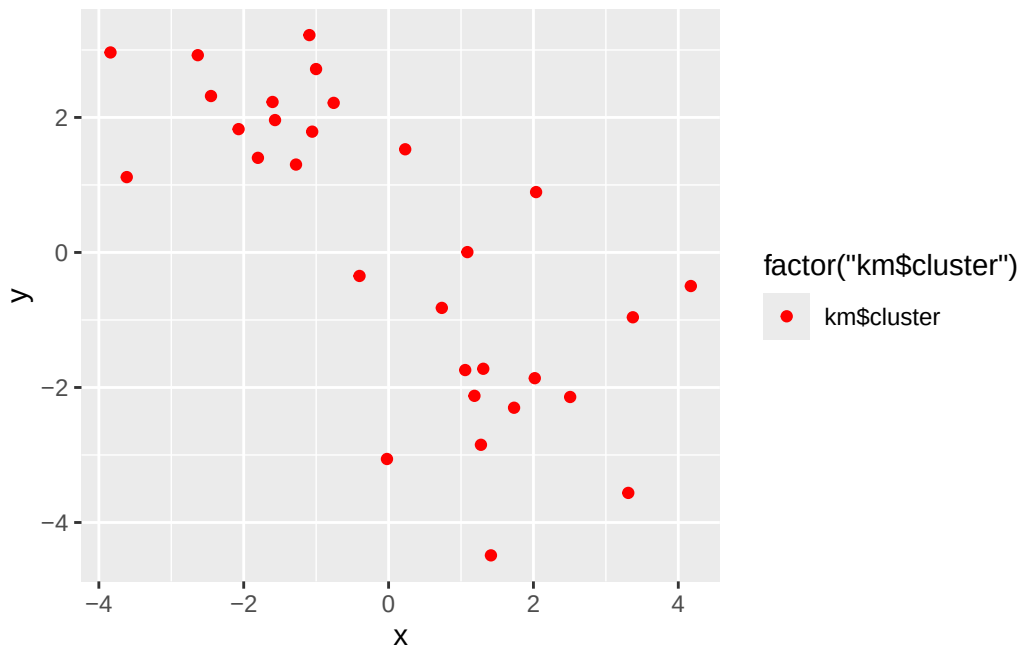
Plot kmeans clustering

```
plot(df, col=c("RED", "BLACK")[km$cluster])  
points(km$centers, col="BLUE", pch="X")
```



Plot with ggplot:

```
library(ggplot2)
ggplot(df) +
  aes(x, y, col=factor("km$cluster")) +
  geom_point() +
  scale_color_manual(values=c("RED", "BLACK"))
```



```
geom_point(data=km$centers, col="BLUE", shape="X", size=3, aes(x, y))
```

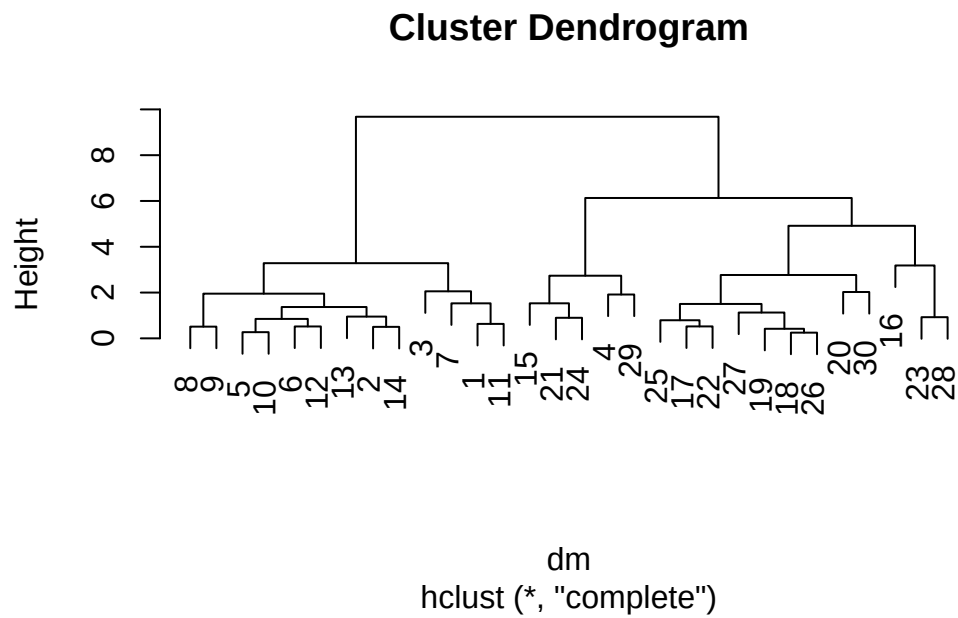
```
mapping: x = ~x, y = ~y
geom_point: na.rm = FALSE
stat_identity: na.rm = FALSE
position_identity
```

Demonstrate the use of `dist()`, `hclust()`, `plot()`, and `cutree()`

```
dm <- dist(df)
```

```
hc <- hclust(dm)
```

```
plot(hc)
```

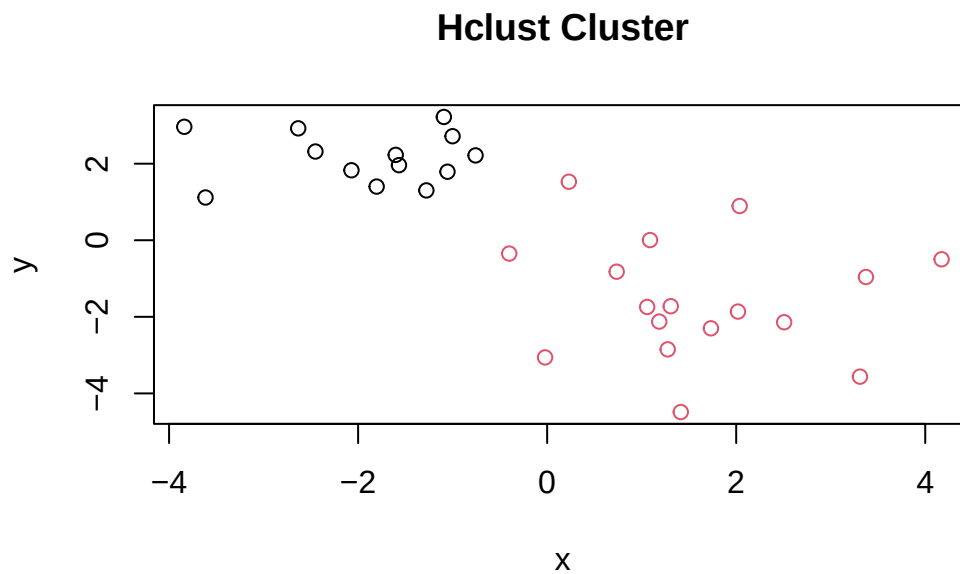


Cut tree into 2 clusters

```
groups <- cutree(hc, k=2)  
print(groups)
```

```
[1] 1 1 1 2 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

```
plot(df, col=groups, main="Hclust Cluster")
```



Lab 07

Data Import

```
url <- "https://tinyurl.com/UK-foods"  
x <- read.csv(url)
```

Q: How many rows and columns are in your new data frame named `x`?
What R functions could you use to answer this questions?

```
dim(x)
```

```
[1] 17  5
```

Checking your data: preview the first 6 rows.

```
head(x)
```

	X	England	Wales	Scotland	N.Ireland
1	Cheese	105	103	103	66
2	Carcass_meat	245	227	242	267
3	Other_meat	685	803	750	586
4	Fish	147	160	122	93
5	Fats_and_oils	193	235	184	209
6	Sugars	156	175	147	139

Hmm, it looks like the row-names here were not set properly as we were expecting 4 columns (one for each of the 4 countries of the UK - not 5 as reported from the `dim()` function).

Here it appears that the row-names are incorrectly set as the first column of our `x` data frame (rather than set as proper row-names). This is very common and sometimes what we want - but not in this case. Lets try to fix this up with the following code, which sets the `rownames()` to the first column and then removes the troublesome first column (with the `-1` column index):

```
rownames(x) <- x[,1]
x <- x[,-1]
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

```
dim(x)
```

```
[1] 17 4
```

Side-note: An alternative approach to setting the correct row-names in this case would be to read the data file again and this time set the `row.names` argument of `read.csv()` to be the first column (i.e. use argument setting `row.names=1`), see below:

```
x <- read.csv(url, row.names=1)
head(x)
```

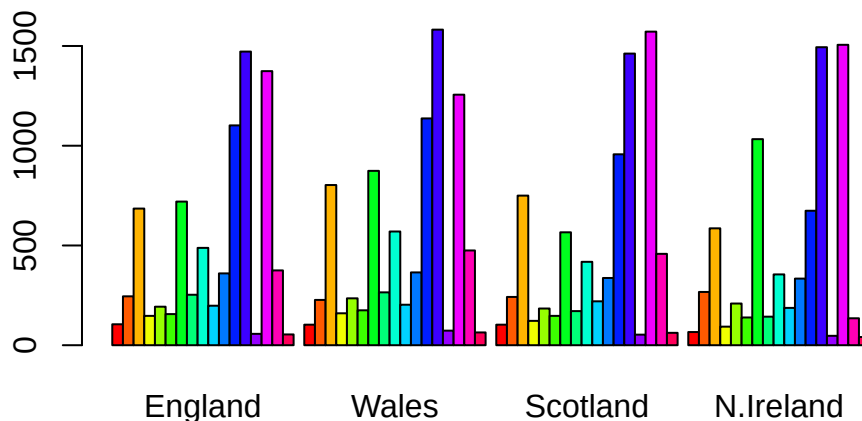
	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

Q2: Which approach to solving the ‘row-names problem’ mentioned above do you prefer and why? Is one approach more robust than another under certain circumstances?

I would prefer the one where we read the csv. This is because it is more simple and generally more used when trying to acquire data from a file. The other approach also creates a whole data frame first, then modify it. This causes more room for mistakes.

Supporting major differences and trends:

```
barplot(as.matrix(x), beside=T, col=rainbow(nrow(x)))
```



Q3: Changing what optional argument in the above barplot() function results in the following plot?

Changing the `beside` function result is what causes the values to be stacked on top. In the barplot above, `beside = 2`, this is why each segment is next to each other. When `beside = false`, the results are stacked on top of each other.

Using the `ggplot2` package for these relatively simple figures is somewhat more convoluted than using “base” R. This is due to the `ggplot` requirement for so-called “long” data rather than the conventional “wide” data format that we currently have. We will talk more about “long” format data a little later in the course when we introduce the `dplyr` package.

```
dim(x)
```

```
[1] 17  4
```

```
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

```
library(tidyr)
x_long <- x |>
  tibble::rownames_to_column("Food") |>
  pivot_longer(cols = -Food,
               names_to = "Country",
               values_to = "Consumption")
```

```
dim(x_long)
```

```
[1] 68  3
```

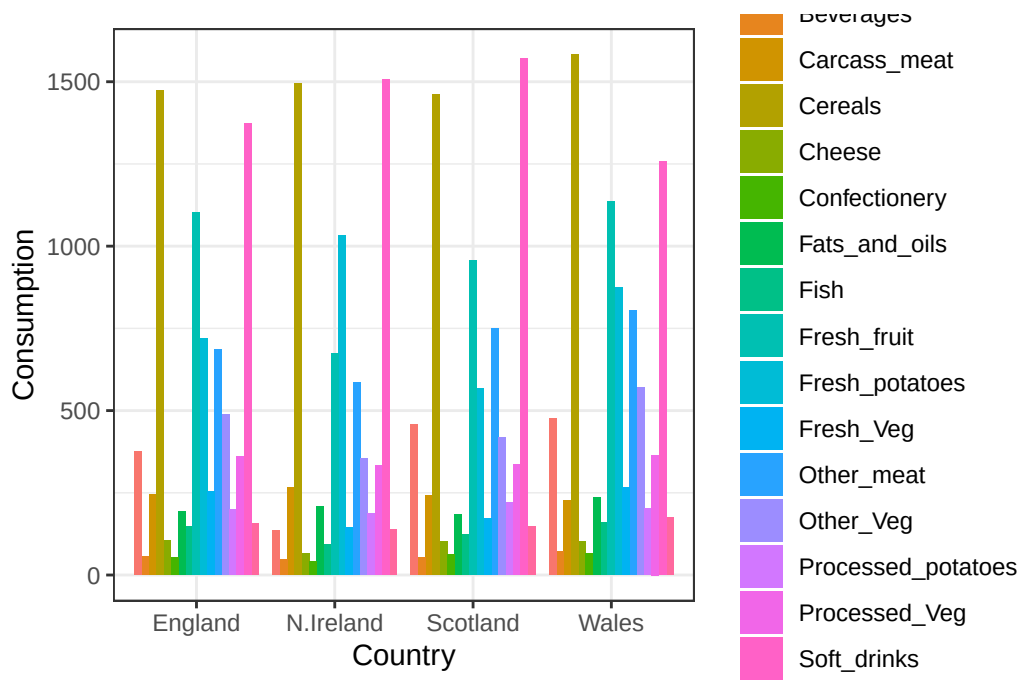
```
head(x_long)
```

```
# A tibble: 6 x 3
  Food          Country Consumption
<chr>         <chr>         <int>
1 Cheese      England         105
2 Carcass_meat England         245
3 Other_meat   England         685
4 Fish         England         147
5 Fats_and_oils England         193
6 Sugars       England         156
```

1	"Cheese"	England	105
2	"Cheese"	Wales	103
3	"Cheese"	Scotland	103
4	"Cheese"	N.Ireland	66
5	"Carcass_meat "	England	245
6	"Carcass_meat "	Wales	227

Create a bar plot:

```
library(ggplot2)
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "dodge") +
  theme_bw()
```

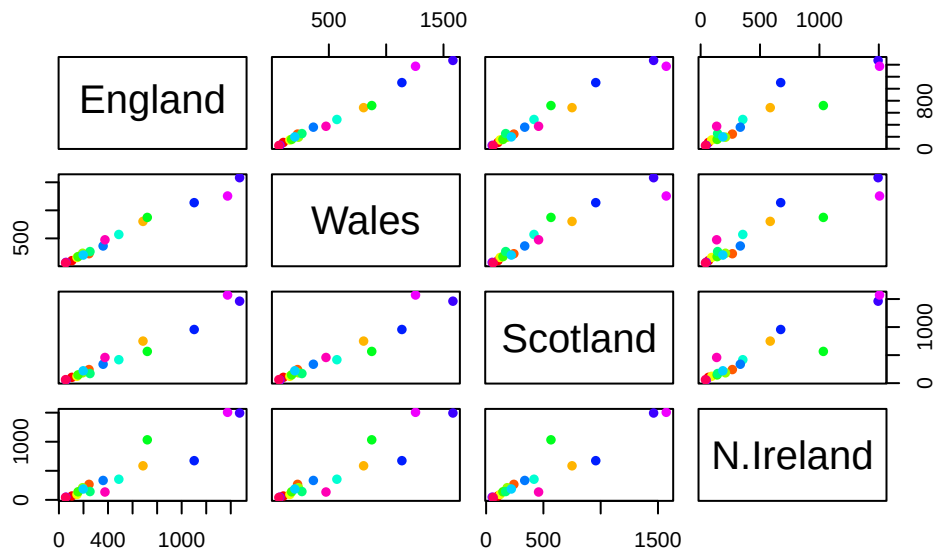


Q4: Changing what optional argument in the above ggplot() code results in a stacked barplot figure?

Through default ggplot has a position argument within its `geom_col()` function. To make the bars stacked on top of each other, you must make `position = stack`. When `position = dodge`, the bars are side by side.

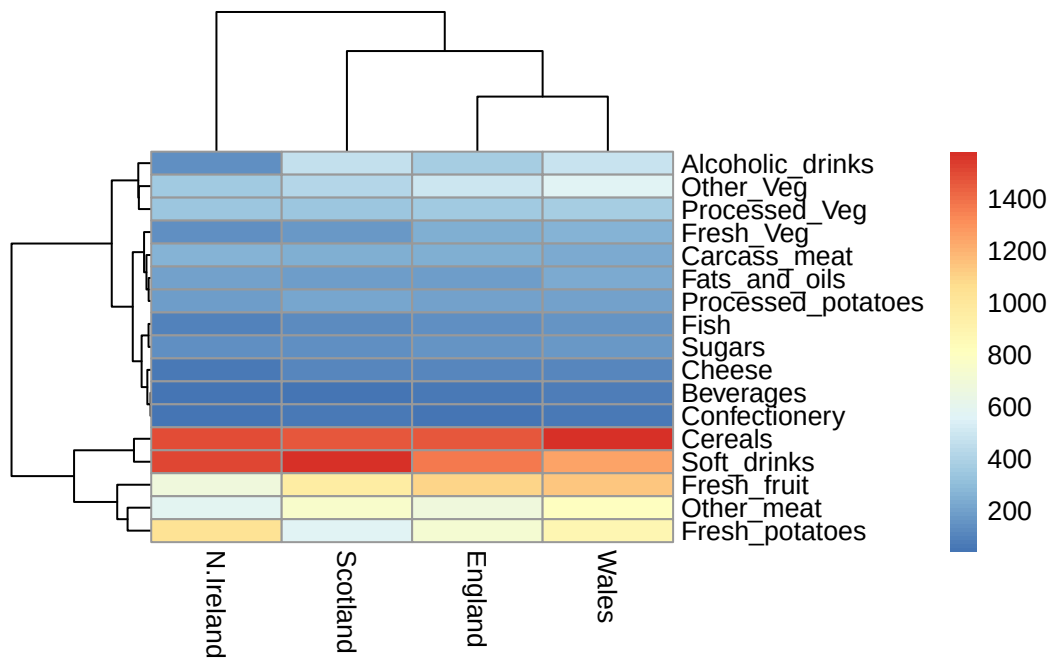
Q5: We can use the `pairs()` function to generate all pairwise plots for our countries. Can you make sense of the following code and resulting figure? What does it mean if a given point lies on the diagonal for a given plot?

```
pairs(x, col=rainbow(nrow(x)), pch=16)
```



```
library(pheatmap)

pheatmap( as.matrix(x) )
```



Q6. Based on the pairs and heatmap figures, which countries cluster together and what does this suggest about their food consumption patterns? Can you easily tell what the main differences between N. Ireland and the other countries of the UK in terms of this data-set?

Based on the map above, it is clear that none of the countries are too far off from each other. When looking at the map even more deeply, it's clear that N. Ireland is slightly more distinct. This is because in some regions like the fresh fruit area, the country had lower values compared to the rest. Northern Ireland seems to have had lower values in regions like other-meat and fresh fruit. Overall the countries seem to have similar food/eating patterns.

PCA to the rescue:

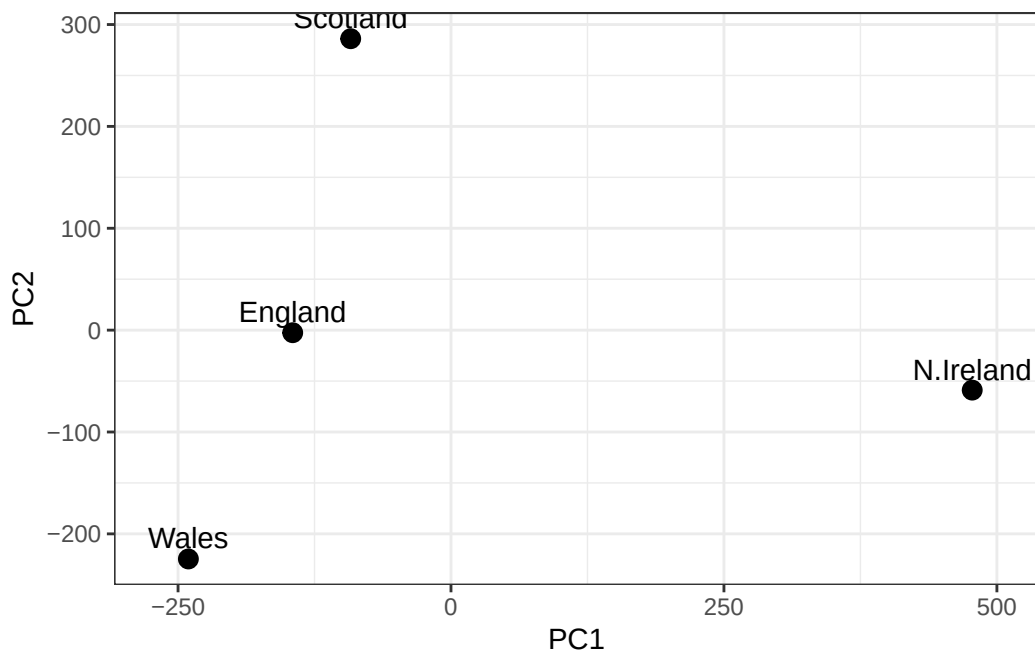
```
pca <- prcomp( t(x))
summary(pca)
```

Importance of components:

	PC1	PC2	PC3	PC4
Standard deviation	324.1502	212.7478	73.87622	3.176e-14
Proportion of Variance	0.6744	0.2905	0.03503	0.000e+00
Cumulative Proportion	0.6744	0.9650	1.00000	1.000e+00

Q7. Complete the code below to generate a plot of PC1 vs PC2. The second line adds text labels over the data points.

```
df <- as.data.frame(pca$x)
df$Country <- rownames(df)
ggplot(pca$x) +
  aes(x = PC1, y = PC2, label = rownames(pca$x)) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



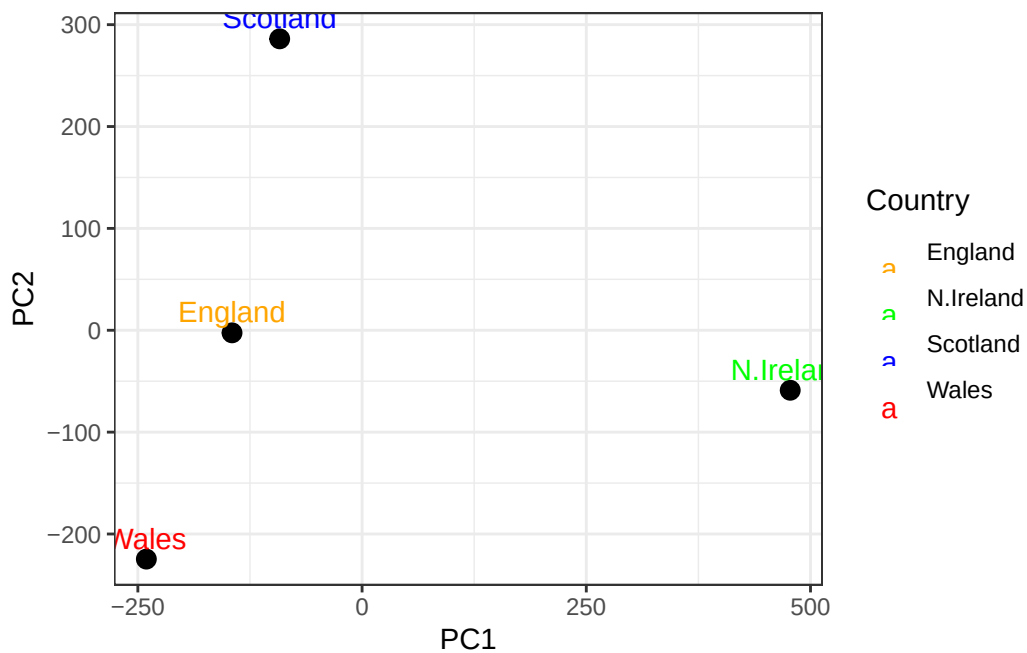
Q8. Customize your plot so that the colors of the country names match the colors in our UK and Ireland map and table at start of this document.

```
df <- as.data.frame(pca$x)
df$Country <- rownames(df)
unique(df$Country)
```

```
[1] "England"    "Wales"      "Scotland"   "N.Ireland"
```

```
ggplot(df) +
  aes(x = PC1, y = PC2, label = Country) +
  geom_point(size = 3) +
  geom_text(aes(col= Country), vjust = -0.5) +
  scale_color_manual(values = c("England"="orange",
                                "Scotland"="blue",
                                "Wales"="red",
                                "N.Ireland"="green")) +

  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



Below we can use the square of `pca$sdev`, which stands for “standard deviation”, to calculate how much variation in the original data each PC accounts for.

```
v <- round( pca$sdev^2/sum(pca$sdev^2) * 100 )
v
```

```
[1] 67 29 4 0
```

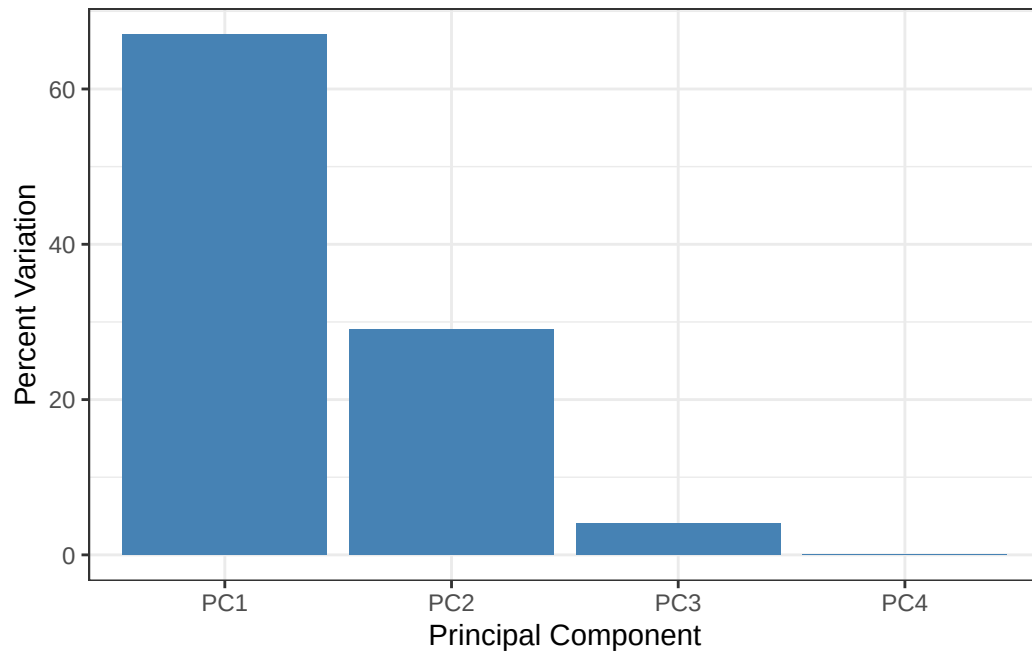
```
z <- summary(pca)
z$importance
```

	PC1	PC2	PC3	PC4
Standard deviation	324.15019	212.74780	73.87622	3.175833e-14
Proportion of Variance	0.67444	0.29052	0.03503	0.000000e+00
Cumulative Proportion	0.67444	0.96497	1.00000	1.000000e+00

This information can be summarized in a plot of the variances (eigenvalues) with respect to the principal component number (eigenvector number), which is given below.

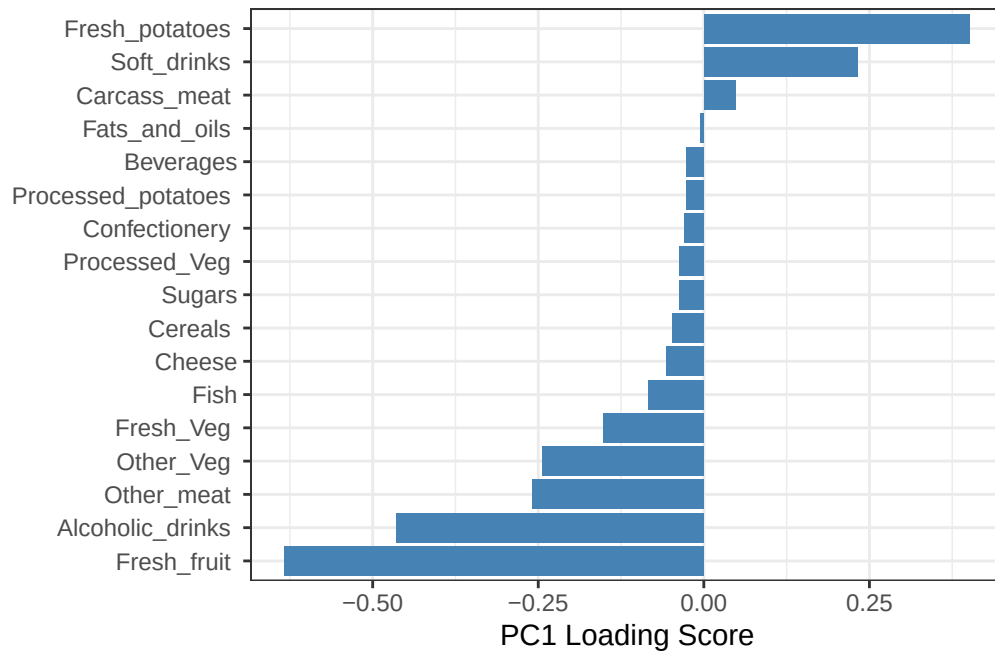
```
variance_df <- data.frame(
  PC = factor(paste0("PC", 1:length(v)), levels = paste0("PC", 1:length(v))),
  Variance = v
)

ggplot(variance_df) +
  aes(x = PC, y = Variance) +
  geom_col(fill = "steelblue") +
  xlab("Principal Component") +
  ylab("Percent Variation") +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 0))
```



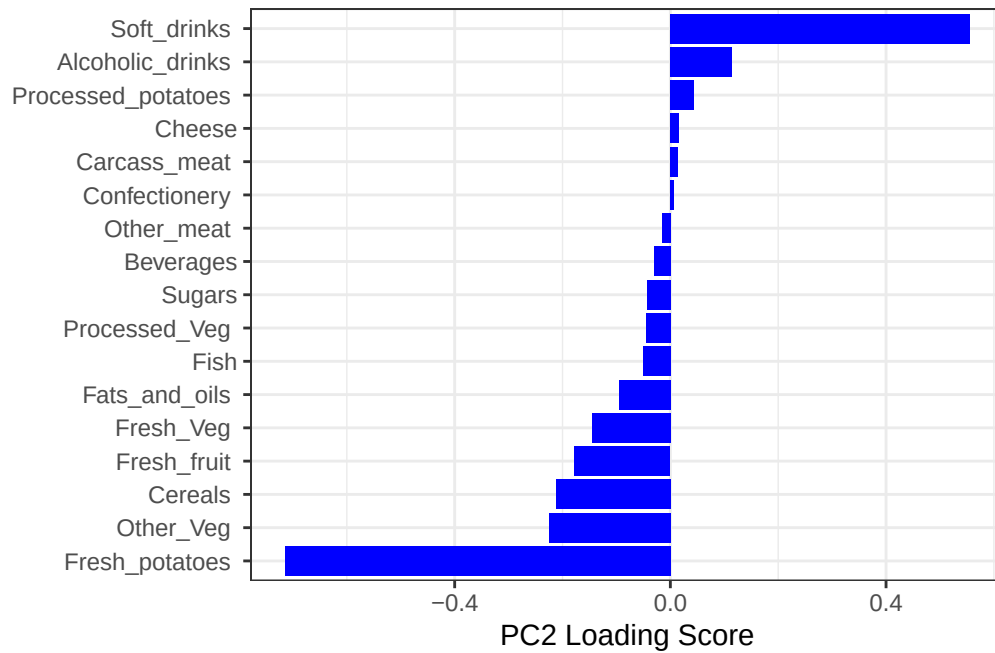
Variable Loadings

```
ggplot(pca$rotation) +  
  aes(x = PC1,  
      y = reorder(rownames(pca$rotation), PC1)) +  
  geom_col(fill = "steelblue") +  
  xlab("PC1 Loading Score") +  
  ylab("") +  
  theme_bw() +  
  theme(axis.text.y = element_text(size = 9))
```

Q9: Generate a similar ‘loadings plot’ for PC2. What two food groups feature prominently and what does PC2 mainly tell us about?

```
ggplot(pca$rotation) +
  aes(x = PC2,
      y = reorder(rownames(pca$rotation), PC2)) +
  geom_col(fill = "blue") +
  xlab("PC2 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```



The two food groups that are frequently featured is soft drinks and fresh potatoes. PC2 helps distinguish Scotland, England, Wales, and Northern Ireland based on their food consumptions. Compared to PC1, PC2 shows the second chunk of variety.